DEVFLOW

Using Claude with DevFlow

Generated from `CLAUDE_AGENT.md` · 2026-10-20

There are two ways to work with Claude here, and they serve different things:

- **The Claude panel**, inside the app — to ask about your requests, build flows and run calls without leaving the screen.
- **Claude Code in the terminal**, connected to DevFlow through an MCP server — for when the work is in the code, but you want the agent to fire the requests already configured here.

Both use the **same CLI** (`claude`) underneath. What changes is who invokes it and what it can see.

---

1. The Claude panel

Where it is

The Claude icon in the sidebar. The panel opens beside the app, with the conversation history.

Signing in

Click the key icon at the top of the panel, or the **Fix sign-in / API key** button that appears next to a credential error. Three routes:

Option	Uses	When it makes sense
Claude subscription	Your Claude subscription	The normal case. Does not consume API credits.
Anthropic Console	API credits, billed per use	If you work with an organisation key
API key	A key pasted into the field	A standalone key, without logging in

The first two open **a real terminal window** running `claude auth login` . That is deliberate: the flow is interactive — it opens a browser and may ask you to paste a code back. Run hidden, it would hang waiting for input nobody can type.

The separate profile (important)

The **"Keep a separate login for DevFlow"** box is checked by default. With it, the app points the CLI at its own config directory:

```
%APPDATA%\com.DevFlow.dev\claude\
```

instead of the global `~/.claude` . The reason: signing in through the panel must not knock out the account you already use in the terminal. With separate profiles you can use a personal account here and keep the other one out there.

The confusing side effect: if you are already signed in to `claude` in the terminal, the panel will still say **"not logged in"** — correctly, because it is looking at a different directory. To reuse that account, **uncheck** the box. The profile then also sees your global settings, MCP servers and `CLAUDE.md` .

To check whether the login worked, the profile's `.claude.json` gains `oauthAccount`, `emailAddress` and `organizationName`. The credential itself lives in `.credentials.json` beside it — that one should not be copied anywhere.

The credential is **per machine and per person**. There is nothing you can provision for a colleague: they need their own subscription, on their own machine.

What the panel can see

When the MCP server is on (section 2), the panel **passes its configuration to the CLI automatically**. You do not need to run `claude mcp add` to use the panel — it opens with the DevFlow tools already available, scoped to `mcp__devflow`.

You can also set a **working directory**: the folder the CLI runs in, and therefore what it can read. Without it, the agent answers about your collections but not about your code.

A limit worth knowing

The panel runs the CLI in `dontAsk` mode: **there is no permission prompt**. Nothing can sit waiting for someone to answer inside a chat panel, so anything not pre-approved is denied and execution continues. In practice the panel's agent does what the DevFlow tools allow, and no more.

---

2. Claude Code in the terminal

Here Claude runs outside the app, and DevFlow joins as an **MCP server**: the CLI gains tools to list and execute your requests.

Turning the server on

Settings → Claude Code (MCP) → Turn on.

The server listens **on 127.0.0.1 only**, and **every call requires the token** shown in the panel. That is not a formality: while it is on, anyone with the token can fire your requests with your environments' credentials. Turn it off when you are not using it.

Connecting

The panel shows the command ready to copy. Run it **once**:

```
claude mcp add --transport http devflow http://127.0.0.1:60401/mcp --header "Authorization: Bearer YOUR_TOKEN"
```

The port is assigned by the system the first time and then **remembered**, precisely so this command keeps working across restarts.

> **If the port changes:** the app warns in the panel and asks you to run the command again. This happens on Windows because Hyper-V/WSL reserves ranges of high ports and re-draws them at every boot — a port the system handed out yesterday can be refused today. When that happens, DevFlow falls back to another port instead of simply failing to start.

Regenerating the token invalidates what you already configured: anyone connected starts getting 401 until they run the command again. That is the button's purpose.

The token

It lives in the system's secure storage — the same place as the GitHub token — not in `localStorage`. It authorises firing any saved request with your real environments, so it deserves at least the care of a password.

---

3. What the agent can do

The tools are the same in both modes.

Read the tree `list_requests` · `list_folders` · `get_request` · `search_requests` · `list_examples` · `get_example` · `list_environments`

Execute `run_request` · `run_collection` · `run_load_test` · `list_load_tests` · `get_oauth2_token`

Read what already happened `get_active_request` · `get_response` · `get_console_logs` · `get_last_run`

`get_last_run` returns the result of the last Collection Runner run — status, assertions and the **response body** of each request. It is what stops the agent asking you to re-run a flow that just ran: a request executed by the Runner does not open a tab, so `get_response` cannot find it.

Write `create_folder` · `create_request` · `duplicate_request` · `rename_item` · `set_request_script`

Reference `get_script_api` — the `pm.*` API available in scripts. Worth calling before asking it to write one. See Scripting.

Execution is the same as the Send button

`run_request` and `run_collection` go through exactly the same path as Send: same `{{variable}}` resolution, same auth, same body sanitisation, same scripts. What the agent fires is not a simulation.

Token steps (MSAL / OAuth2)

A request with **OAuth 2.0** authorization and an **empty URL** is not broken: that is how the app stores an OAuth2 configuration. The token comes from the authorization flow — the **Get new access token** button, or `get_oauth2_token` — and lands in the environment variable the *Access Token* field points at. There is nothing to send, which is why there is no URL.

`get_request` marks these with `tokenStep: true` and explains what to do, and `run_request` refuses to send them, pointing at the right tool. Without that the agent reads "empty URL", concludes the request is defective, and suggests filling it in or deleting the step — which would break every flow that depends on the variable.

In the **Collection Runner** the step is executed as a token fetch rather than a send: it reuses the token already in the variable while more than 5 minutes of validity remain, and opens the login window only when a refresh is needed. The token goes into that run's environment, so the following steps find it.

Secrets come back masked

`get_request` **redacts** what looks like a credential: tokens, passwords, the OAuth2 `client_secret`, and headers whose name suggests a secret. A variable reference (`{{token}}`) is preserved on purpose — it is not a secret, and it is exactly what the agent needs to see to understand where the value comes from.

That is why `duplicate_request` exists: copying an OAuth2 request by reading and recreating it would produce a copy without the credentials, silently broken. Duplication happens inside the app, where the real

values are, and they never pass through the model.

What it does not do

Nothing goes to GitHub. Creating, renaming or editing through the agent saves **locally only** — pushing stays an explicit action of yours. This is deliberate: an agent iterating over scripts would generate a flood of commits.

---

4. Which one, when

Situation	Use
"Why is this request returning 400?"	Panel
"Build a cancellation flow with refund"	Panel
"Run the collection and tell me what broke"	Either
You are working in the code and need to hit an API	Terminal
You want the agent to read the whole repository	Terminal

The rule of thumb: **if the work is about the requests, use the panel; if it is about the code, use the terminal** and let DevFlow be the source of the calls.

---

5. When something goes wrong

"Not logged in · Please run /login" in the panel The CLI is installed and ran — it just has no credential *in that profile*. See the separate-profile section: most of the time it is the "Keep a separate login for DevFlow" box being checked while your login lives in the global profile.

"Credit balance is too low" You signed in through **Anthropic Console**, which bills API usage. For a subscription, sign out and sign in through **Claude subscription**.

"Invalid API key" even with the account configured Some `ANTHROPIC_API_KEY` in the machine's environment is beating the signed-in account — the CLI prefers it. The app removes that variable in account mode, but it is worth checking whether it exists in your environment.

`Failed to bind 127.0.0.1:NNNNN` (error 10013) The port is **reserved** by the system, not occupied. To see the ranges:

```
netsh interface ipv4 show excludedportrange protocol=tcp
```

The app falls back to another port on its own; after that, run `claude mcp add` again with the new port.

The agent says a tool does not exist The list above is what exists. If an operation is missing it has to be added in `src/mcp/mcpTools.ts` — there is no way for the agent to work around it.